

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a5ebe0cfe9685049b.jsonl`  
- SHA-256: `fbd752df16be7fce9eae2f3c8b71bfda9c620f13c6606f7b9c1a1d8a4b4d1f5`  
- Source modified: `2026-06-10T06:14:56+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:12:46.621Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("eng-pipeline") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"Re-ingesting a video that NOW fails validation cascade-deletes all its previously good segments before returning early","severity":"high","area":"API / DB lifecycle","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py:112,369","evidence":"Both the API path (main.py:112, inside the failures branch) and the CLI path (main.py:369, unconditionally before the failure check) call db.add_video(video_id, path, 'failed', ...). add_video is INSERT OR REPLACE (database.py:107); with PRAGMA foreign_keys=ON the conflict-replace deletes the old videos row and CASCADE-deletes every play_segment, event, and candidate_segment for that video_id. The early return then leaves the video with status='failed' and zero segments.", "why_it_matters":"A transient ffmpeg hiccup, a tightened validation threshold in config, or re-POSTing a video on a machine where ffmpeg dropped off PATH permanently destroys prior indexing results (including the candidate_segments training corpus rows) with nothing replacing them. CODE_MAP documents the add_video cascade only as a benign 're-ingest replaces prior results' note (and spec.yaml's reingest_replaced_prior is severity info); the failed-validation wipe is strictly destructive and matters MORE than documented. Confirmed-and-escalated CODE_MAP gotcha."}

Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:12:55.006Z)

```

80
81     class CompileRequest(BaseModel):
82         video_id: str
83         segment_ids: List[int]
84         output_filename: str
85
86     # --- API Endpoints ---
87     @app.get("/api/config")
88     def get_config():
89         return global_config
90
91     @app.post("/api/process")
92     def process_video(req: ProcessRequest):
93         if not os.path.exists(req.video_path):
94             raise HTTPException(status_code=404, detail=f"Video file not found at
95 '{req.video_path}'")
96
97         video_id = compute_video_id(req.video_path)
98         was_reingest = db_instance.get_video(video_id) is not None
99
100        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
101        logger.info(f"Running validations for {req.video_path}...")
102        val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
103        failures = [r for r in val_results if not r.passed]
104
105        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
106        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
107        seg_name = req.segmenter or default_seg
108        segmenter_actual = None
109        segmentation_ran = False
110        response: Optional[Dict[str, Any]] = None
111        try:
112            if failures:
113                db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
114                response = {
115                    "video_id": video_id,
116                    "status": "failed",
117                    "message": "Video failed validation checks.",
118                    "results": [r.dict() for r in val_results],
119                }
120            return response
121
122        # 2. Run segmenter & indexer
123        print(f"[API] Video passed checks. Segmenting and indexing...")
124        db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
125
126        segmenter_cls = segmenter_registry.get(seg_name)
127        if not segmenter_cls:
128            available = list(segmenter_registry.list_available().keys())
129            raise HTTPException(
130                status_code=400,
131                detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
132            )
133
134        print(f"[API] Using segmenter: {seg_name}")
135        segmenter_actual = seg_name
136        segmenter = segmenter_cls(db_instance, global_config)
137        result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)

```

```

137         segmentation_ran = True
138
139     if isinstance(result, Failure):
140         print(f"[API] Segmenter failed: {result.message}")
141         response = {
142             "video_id": video_id,
143             "status": "failed",
144             "message": f"Segmenter '{seg_name}' failed: {result.message}",
145             "results": [r.dict() for r in val_results],
146             "play_segments": [],
147         }
148         return response
149
150     segments = result.value if hasattr(result, "value") else result
151
152     response = {
153         "video_id": video_id,
154         "status": "success",
155         "message": f"Successfully indexed {len(segments)} play segments using
156 '{seg_name}' segmenter.",
157         "results": [r.dict() for r in val_results],
158         "play_segments": segments,
159     }
160     return response

```

3. user (2026-06-10T06:12:55.882Z)

```

340     global global_config, db_instance
341     global_config = load_app_config() # returns typed AppConfig
342
343     # The CLI --output-dir overrides the configured output directory. It now lives
344     # on the typed model (OutputConfig.output_dir), so every consumer ? including
345     # /api/compile ? reads the same value instead of a write-only runtime hack.
346     global_config.output.output_dir = args.output_dir
347     os.makedirs(global_config.output.output_dir, exist_ok=True)
348
349     db_path = os.path.join(global_config.output.output_dir,
350 global_config.output.db_name)
351     db_instance = Database(db_path)
352
353     # CLI processing mode (no web UI needed)
354     if args.video_path and not args.server:
355         print(f"[CLI] Processing video file: {args.video_path}")
356         if not os.path.exists(args.video_path):
357             print(f"[FAIL] Video file not found: {args.video_path}")
358             return
359
360         video_id = compute_video_id(args.video_path)
361         was_reingest = db_instance.get_video(video_id) is not None
362
363         # Validate (with-context variant surfaces ffmpeg_meta for the report)
364         print(f"[CLI] Validating...")
365         val_results, val_context = registry.run_all_with_context(args.video_path,
366 global_config)
367         failures = [r for r in val_results if not r.passed]
368
369         # Save video status
370         status = "failed" if failures else "processed"
371         db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
372 val_results])
373
374     seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
375 "default_segmenter", "motion")
376     segmenter_actual = None
377     segmentation_ran = False

```

```

374         try:
375             print("\n" + "="*40)
376             print("VALIDATION SUMMARY")
377             print("="*40)
378             for r in val_results:
379                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
380                 print(f"{status_symbol} {r.validator_name}: {r.message}")
381             print("="*40 + "\n")
382
383             if failures:
384                 print("[FAIL] Video failed checks. Indexing halted.")
385                 return
386
387             # Index
388             segmenter_cls = segmenter_registry.get(seg_name)
389             if not segmenter_cls:
390                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
391                 return
392
393             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
394             segmenter_actual = seg_name
395             segmenter = segmenter_cls(db_instance, global_config)
396             result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
397             segmentation_ran = True
398
399             if isinstance(result, Failure):
400                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
401                 print(f"[FAIL] Indexing halted for video.")
402                 segments = []
403             else:
404                 segments = result.value if hasattr(result, "value") else result
405                 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
406             finally:
407                 # Always emit the per-video observability report. Never raises.
408                 paths = emit_indexer_report(
409                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
410                     val_results=val_results, val_context=val_context,
411                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
412                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
413                 )
414             if paths:
415                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
416                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
417                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
418             return
419

```

4. user (2026-06-10T06:13:03.168Z)

```

1     import sqlite3
2     import json
3     import os
4     import logging
5     from typing import List, Dict, Any, Optional
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13

```

```

14     def _get_connection(self):
15         conn = sqlite3.connect(self.db_path)
16         conn.row_factory = sqlite3.Row
17         # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
18         conn.execute("PRAGMA foreign_keys = ON")
19         return conn
20
21     def _init_db(self):
22         """Initializes tables for videos, play_segments, and events."""
23         with self._get_connection() as conn:
24             cursor = conn.cursor()
25
26             # Videos Table
27             cursor.execute("""
28                 CREATE TABLE IF NOT EXISTS videos (
29                     id TEXT PRIMARY KEY,
30                     filepath TEXT NOT NULL,
31                     processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
32                     status TEXT NOT NULL,
33                     validation_results TEXT
34                 )
35             """)
36
37             # Play Segments Table (Extensible metadata JSON)
38             cursor.execute("""
39                 CREATE TABLE IF NOT EXISTS play_segments (
40                     id INTEGER PRIMARY KEY AUTOINCREMENT,
41                     video_id TEXT NOT NULL,
42                     start_time REAL NOT NULL,
43                     end_time REAL NOT NULL,
44                     duration REAL NOT NULL,
45                     server TEXT,
46                     receiver TEXT,
47                     metadata TEXT,
48                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
49                 )
50             """)
51
52             # Events Table
53             cursor.execute("""
54                 CREATE TABLE IF NOT EXISTS events (
55                     id INTEGER PRIMARY KEY AUTOINCREMENT,
56                     segment_id INTEGER NOT NULL,
57                     timestamp REAL NOT NULL,
58                     type TEXT NOT NULL,
59                     player TEXT,
60                     details TEXT,
61                     FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
62                     CASCADE
63                 )
64             """)
65
66             # Versioned migrations live here. PRAGMA user_version is the schema
67             # contract ? bump it for every change and add an ordered `if version < N`
68             # block below. Tests assert the expected version, so accidental drift fails
69             CI.
70             version = cursor.execute("PRAGMA user_version").fetchone()[0]
71
72             if version < 1:
73                 # v1: raw candidate detections (pre-confirmation), detector-agnostic.
74                 # Common fields are columns; detector-specific metrics go in `stats`
75                 JSON,
76                 # so a new segmenter reuses this table by setting its own
77                 `detector`/`stats`.
78                 cursor.execute("""

```

```

75         CREATE TABLE IF NOT EXISTS candidate_segments (
76             id INTEGER PRIMARY KEY AUTOINCREMENT,
77             video_id TEXT NOT NULL,
78             detector TEXT NOT NULL,
79             chunk_index INTEGER,
80             start_time REAL NOT NULL,
81             end_time REAL NOT NULL,
82             duration REAL NOT NULL,
83             confidence REAL,
84             stats TEXT,
85             detection_params TEXT,
86             confirmed_segment_id INTEGER,
87             human_label TEXT,
88             notes TEXT,
89             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
90             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
91             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
92         )
93     """
94     cursor.execute("""
95         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
96         ON candidate_segments(video_id, detector)
97     """)
98     cursor.execute("PRAGMA user_version = 1")
99     # future: if version < 2: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 2
100
101     conn.commit()
102
103     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
104         try:
105             with self._get_connection() as conn:
106                 conn.cursor().execute(
107                     "INSERT OR REPLACE INTO videos (id, filepath, status,
validation_results) VALUES (?, ?, ?, ?)",
108                     (video_id, filepath, status, json.dumps(validation_results))
109                 )
110                 conn.commit()
111             return True
112         except Exception as e:
113             logger.error(f"Failed to add video: {e}")
114             return False
115
116     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
117         try:
118             with self._get_connection() as conn:
119                 cursor = conn.cursor()
120                 cursor.execute(
121                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
122                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
123                 )
124                 conn.commit()
125                 return cursor.lastrowid
126         except Exception as e:
127             logger.error(f"Failed to add play segment: {e}")
128             return None
129
130     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:

```

```

131         try:
132             with self._get_connection() as conn:
133                 conn.cursor().execute(
134                     "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
135                     (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
136             )
137             conn.commit()
138             return True
139         except Exception as e:
140             logger.error(f"Failed to add event: {e}")
141             return False
142
143     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
144                               chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
145                               stats: Optional[Dict[str, Any]] = None,
146                               detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
147         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
148         detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
149         try:
150             with self._get_connection() as conn:
151                 cursor = conn.cursor()
152                 cursor.execute(
153                     """INSERT INTO candidate_segments
154                     (video_id, detector, chunk_index, start_time, end_time, duration,
155                     confidence, stats, detection_params)
156                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
157                     (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
158                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
159                 )
160             conn.commit()

```

5. assistant (2026-06-10T06:13:17.672Z)

Mechanics confirmed in code. Now let me check the documentation counter-evidence ? CODE_MAP gotchas, DEFERRED_HARDENING_PLAN, and the spec.yaml mention.

6. user (2026-06-10T06:13:19.669Z)

```

docs\CODE_MAP.md:28: -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
docs\CODE_MAP.md:83: - `::process_video` `@POST /api/process` -> validate -> add_video ->
segment -> `finally:` always `emit_indexer_report` (never raises). `::compile_highlights` `@POST
/api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`.
`::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs
`indexing.motion_threshold` (dual model/dict read).
docs\CODE_MAP.md:173:[Omitted long matching line]
docs\CODE_MAP.md:176:[Omitted long matching line]
docs\CODE_MAP.md:177:[Omitted long matching line]
docs\CODE_MAP.md:178:[Omitted long matching line]
docs\CODE_MAP.md:214:[Omitted long matching line]
docs\CODE_MAP.md:250:- **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a
`video_id` cascade-deletes its segments. Use `clear_video_data` deliberately. `_get_connection`
MUST re-issue `PRAGMA foreign_keys=ON` per connection.
docs\PLATFORM_ARCHITECTURE.md:514:- **Right to erasure / retention:** per-tenant, per-athlete
delete must cascade across DB +
docs\PLATFORM_ARCHITECTURE.md:515: storage artifacts (the `ON DELETE CASCADE` discipline
already in `database.py` extends here).

```

docs\archives\decisions\DECISIONS.md:134: curation status (deleting would cascade-delete the valuable labels).

7. user (2026-06-10T06:13:20.467Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md-243-
README.md-244-### Q10: Can I trust the per-segment metadata from the `motion` segmenter?
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md-245-[Omitted long context line]
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md-246-
README.md-247-### Q11: I re-ran a video and my previous results changed/disappeared. Why?
README.md:248:Videos are keyed by the **MD5 of the file's bytes**. Re-processing the *same* file
is treated as a fresh ingest: `add_video` is `INSERT OR REPLACE`, and the foreign-key cascade
**deletes the previous play/candidate segments** before writing the new ones. So the earlier
results are intentionally replaced, not merged. (A `reingest_replaced_prior` note in the report
flags this.) Note also that re-encoding or re-downloading a video changes its bytes ? a new MD5
? a separate record.
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md-249-
README.md-250-### Q12: What is the `*.report.json` / `*.report.md` / `*.summary.md` file next to
my video?
README.md-251-Every processing run now writes a **per-video observability report** next to the
video (or to `report.output_dir` if set in `config.json`):
README.md-252-- **`<name>.report.json`** ? machine-readable: every stage, metric, and finding.
The source of truth; diff two runs to see what changed.
README.md-253-- **`<name>.report.md`** ? the **technical** report: validation results,
processing stages, and **findings/warnings**. Each warning names the problem and cites the FAQ
entry to read. **Start here when a run looks wrong.**
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-115-      you
ran the detector you meant.
backend\reporting\spec.yaml-116-      faq_ref: "README#Q7"
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-117-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-118- # Re-
processing an already-indexed video replaced its prior segments (INSERT OR
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-119- # REPLACE
cascades). Note it so a changed result isn't mistaken for instability.
backend\reporting\spec.yaml:120: - code: reingest_replaced_prior
backend\reporting\spec.yaml-121-     severity: info
backend\reporting\spec.yaml-122-     when:
backend\reporting\spec.yaml-123-       - { path: "stages.segmentation.details.was_reingest", op:
eq, value: true }
backend\reporting\spec.yaml-124-     message: >-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml-125-      This
video was already in the database; re-processing REPLACED its previous
--
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md-173-[Omitted long context
line]
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md-174-
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md-175-### 2.8 Reporting /
observability ? (per-video, soft-dep `obsreport`)
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md-176-[Omitted long context
line]
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md-177-[Omitted long context
line]
docs\CODE_MAP.md:178:[Omitted long matching line]
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md-179-
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md-180----
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md-181-
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md-182-## 3. DATA CONTRACTS
(canonical shapes ? one place)
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md-183-
--
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-114-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-115-
tests\test_reporting_harvest.py-116-def test_reingest_info_flag(monkeypatch):
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-117-
```

```
monkeypatch.setenv("GEMINI_API_KEY", "x")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-118-   env
= _record(was_reingest=True)
tests\test_reporting_harvest.py:119:   assert "reingest_replaced_prior" in _codes(env)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-120-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-121-
tests\test_reporting_harvest.py-122-def test_validation_failure_halts_and_flags(monkeypatch):
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-123-
monkeypatch.setenv("GEMINI_API_KEY", "x")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py-124-   env
= _record(
```

8. user (2026-06-10T06:13:47.795Z)

```
115
116     - The ShuttleSet corpus **tops out at 1080p** (no 4K); "best available" resolves to
1080p.
117     - Authenticating with a **YouTube Premium** account is required for reliable,
unthrottled
118     downloads ? anonymous downloads are throttled (~1.3 MiB/s) and **non-deterministic**
119     (identical commands variously yielded 1080p DASH, 1080p HLS, or a 144p fallback).
120     - **Chrome** cookies can't be read on Windows (app-bound encryption / DPAPI, yt-dlp
121     #10927); **Firefox** cookies read cleanly even while open.
122     - The `web`/`web_safari` player clients are forced into **SABR streaming**, which strips
123     download URLs and silently drops to 144p (yt-dlp #12482). yt-dlp's `default` client
124     serves real downloadable 1080p DASH with auth cookies.
125     - Bulk bursts trip YouTube **rate-limiting** (a 22-video burst failed all at once,
though
126     each worked in isolation).
127
128     ### Decision
129
130     1. **Always fetch the best resolution available** (`bv*+ba/b -S res,...`, merged to
MP4);
131     ffpprobe the result and store the *actual* width/height/fps ? never assume.
132     2. **Upgrade in place, never clean-slate:** the `fetch` command re-downloads a video and
133     updates only `local_path`/`resolution`/`fps`, preserving every rally annotation and
134     curation status (deleting would cascade-delete the valuable labels).
135     3. Use **authenticated cookies** (`--cookies-from-browser firefox` / `--cookies file`)
and
136     the **`default` player client** ; enable the EJS challenge solver (deno).
137     4. **Throttle bulk runs** (`--sleep` + per-video `--retries`/backoff) and treat a
138     below-min-height download as a **failure** (discard, don't commit) so a flaky low-res
139     result never overwrites a good entry.
140
141     ### Consequences
142
143     - All 22 fetchable ShuttleSet matches are now true 1080p (one match, shuttle12, has
no
144     source URL in the catalog and remains video-less but keeps its annotations).
145     - Acquisition is reproducible and resumable; the ~26 GB corpus is kept outside git.
146     - Future non-YouTube or 4K sources are supported by the same code (optional
`max_height`).
147
148     ---
149
```

9. user (2026-06-10T06:13:49.568Z)

No matches found

10. user (2026-06-10T06:14:23.641Z)

segments after REPLACE: 0

candidates after REPLACE: 0
video status: failed

11. user (2026-06-10T06:14:52.641Z)

Structured output provided successfully

12. assistant (2026-06-10T06:14:56.246Z)

Verification complete ? structured output returned.